

Computer Vision (CAP4410) Assignment 1

Lukas Kelk

Computer Engineer

Department of Computer Science
Florida Polytechnic University, Florida, USA

January 2025

CONTENTS

CONTENTS	ii
LIST OF FIGURES	iii
1 INTRODUCTION	1
1.1 LOADING THE GIVEN IMAGE AND INTIAL HISTOGRAMS ON THE SCREEN	1
1.2 SCROLL BAR IMPLEMENTATION	3
1.3 SAVE AND QUIT FUNCTION.	5

LIST OF FIGURES

Figure 1.1	Just Images and Histograms	3
Figure 1.2	Added Scroll Bars	4
Figure 1.3	Showed Save Functionality	6

1 INTRODUCTION

This project implemented basic image processing functionality using OpenCV. These included loading and displaying the original image, generating histograms of both the original and previewed images, and interactive scroll bars to dynamically adjust brightness and contrast. It included a saving feature to save the processed image for later use with your adjusted settings intact upon restart. The implementation gave hands-on experience with image manipulation, displaying histograms, and the development of interactive interfaces focused on practical computer vision applications.

1.1 LOADING THE GIVEN IMAGE AND INTIAL HISTOGRAMS ON THE SCREEN

To begin, we must load the given information and histograms into our program.

```
iimageLeft = cv2.imread('dog.bmp')
adjusted = imageLeft.copy()

#make window
cv2.namedWindow(WINDOWNAME)

#show default image
stacked = np.hstack((imageLeft, adjusted))

#generate scaled histograms for the original image
hist_left = generateScaledHistogram(imageLeft)
hist_adjusted = generateScaledHistogram(adjusted)

#stack the images and histograms initially
stacked = np.vstack((stacked, np.hstack((hist_left, hist_adjusted))))

#show the stacked image with histograms
cv2.imshow(WINDOWNAME, stacked)
```

Initially, we load the image 'dog.bmp' into a variable called imageLeft, this will be our reference image for performing filtering. Then we duplicate the image so that we have one to filter, this will be called adjusted. now we start up the window so that we can have an active display. Then we create a stack that allows us to display the two images side by side, assign it to one variable. We can then generate the histograms using the functions which return images of the completed histograms.(See Below for Histogram generation Function) After generating those two images we can stack them horizontally and then vertically with the raw images to get our final display of 2 images and 2 histograms.

The Histogram generation function is shown below.

```
def generateScaledHistogram(image):  
    #convert the image to grayscale for histogram calculation  
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  
  
    #calculate histogram  
    hist = cv2.calcHist([gray], [0], None, [256], [0, 256])  
  
    #Normalize the histogram to fit it in the image window  
    hist = hist / hist.max() #Normalize to the maximum value for scaling  
    #Create a blank image to draw the histogram  
    hist_height = image.shape[0]  
    hist_width = 256  
    hist_image = np.zeros((hist_height, hist_width, 3), dtype=np.uint8)  
  
    #Draw the histogram on the blank image  
    for i in range(1, 256):  
        cv2.line(  
            hist_image,  
            (i-1, hist_height - int(hist[i-1] * hist_height)),  
            (i, hist_height - int(hist[i] * hist_height)),  
            (255, 255, 255), 1  
        )  
  
    #Resize the histogram image to the same size as the original image  
    scaled = cv2.resize(hist_image, (image.shape[1], image.shape[0]))  
  
    return scaled
```

The image is passed to the function where it is then converted to grayscale to represent in the histogram, the histogram is then generated from the image. The values are then normalized. We can then create a blank histogram with the dimensions of the original image so we can show it in the window. We can then fill the blank histogram with our normalized values from the scaled image. the scaled image is then resized and returned.

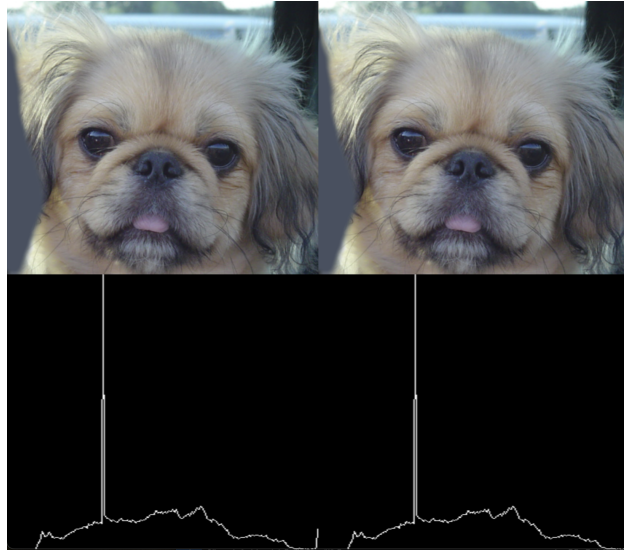


Figure 1.1. Just Images and Histograms

1.2 SCROLL BAR IMPLEMENTATION

The scroll bar for controlling Brightness and Contrast for the picture was implemented fairly simply.

```
#create sliderbar for brightness and contrast
cv2.createTrackbar('Brightness', WINDOWNAME, 128, 255, adjust_brightness)
cv2.createTrackbar('Contrast', WINDOWNAME, 128, 255, adjust_contrast)
```

The createTrackbar option creates trackbars that pass the values of the trackbars into two functions. adjust brightness and adjust contrast.

```
def adjust_brightness(val):
    global lastConst
    global lastBright
    global adjusted

    lastBright = (val / 128)

    adjusted = cv2.convertScaleAbs(imageLeft,
                                   alpha=lastBright, beta=lastConst)

    stacked = np.hstack((imageLeft, adjusted))

    #generate scaled histograms for original and adjusted images
    hist_left = generateScaledHistogram(imageLeft)
```

```
hist_adjusted = generateScaledHistogram(adjusted)

#stack the images and histograms
stacked_with_hist = np.vstack((stacked,
np.hstack((hist_left, hist_adjusted))))

cv2.imshow(WINDOWNAME, stacked_with_hist)
```

The function implementation is fairly simple, to adjust either the brightness or contrast we can use the function `convertScaleABS` adjusting the alpha value in that function it will adjust the brightness of the image and if we adjust beta we will adjust the contrast. We can then save the new image as a variable to load into the program. We then generate the histograms just like before and replace the current images with the new ones we just generated. The adjust contrast function does the same but adjusts the beta value.

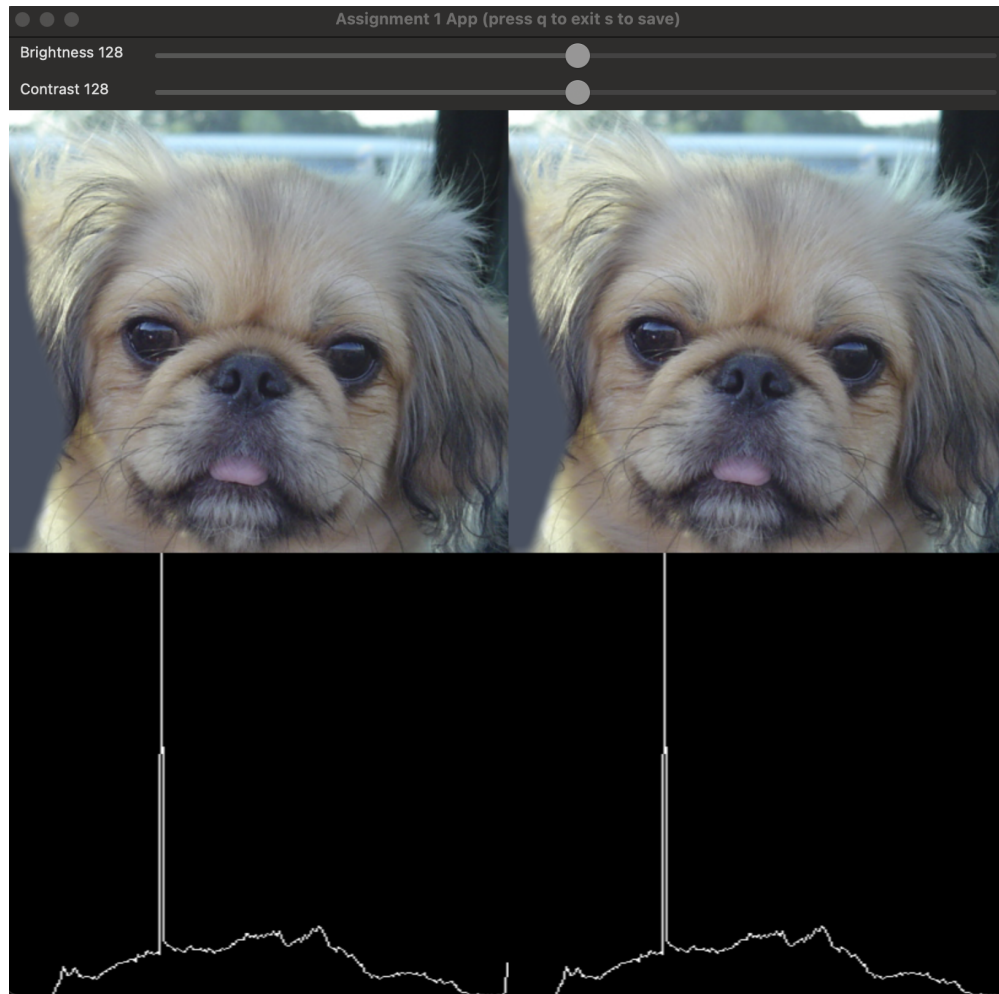


Figure 1.2. Added Scroll Bars

1.3 SAVE AND QUIT FUNCTION

The program also features both a save and a quit function

```
while True:
    if cv2.waitKey(1) == ord('q'):
        break
    if cv2.waitKey(1) == ord('s'):
        # Save the adjusted image
        cv2.imwrite('dog-modified.bmp', adjusted)

        #reload the saved image
        as the new 'imageLeft' for further adjustments
        imageLeft = cv2.imread('dog-modified.bmp')
        adjusted = imageLeft.copy()

        #update the display
        stacked = np.hstack((imageLeft, adjusted))
        hist_left = generateScaledHistogram(imageLeft)
        hist_adjusted = generateScaledHistogram(adjusted)
        stacked_with_hist = np.vstack((stacked,
                                         np.hstack((hist_left, hist_adjusted))))
        cv2.imshow(WINDOWNAME, stacked_with_hist)
```

When the user clicks S the adjusted image on the right overwrites the image on the left and now it becomes the reference image to be adjusted. The program will also save the photo as 'dog-modified.bmp'. If the user clicks q the program will exit when restarted it will load the original loaded image

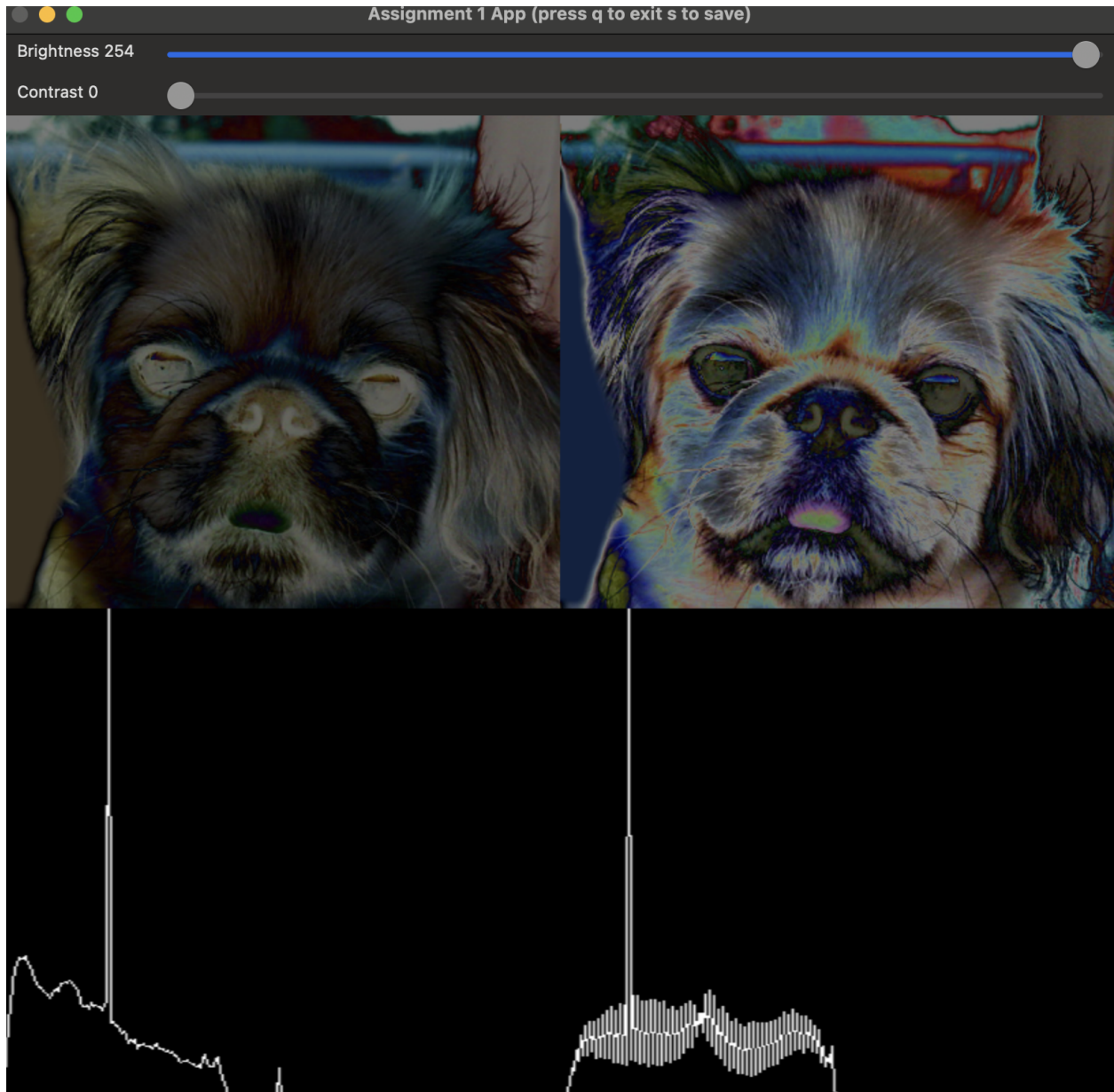


Figure 1.3. Showed Save Functionality